

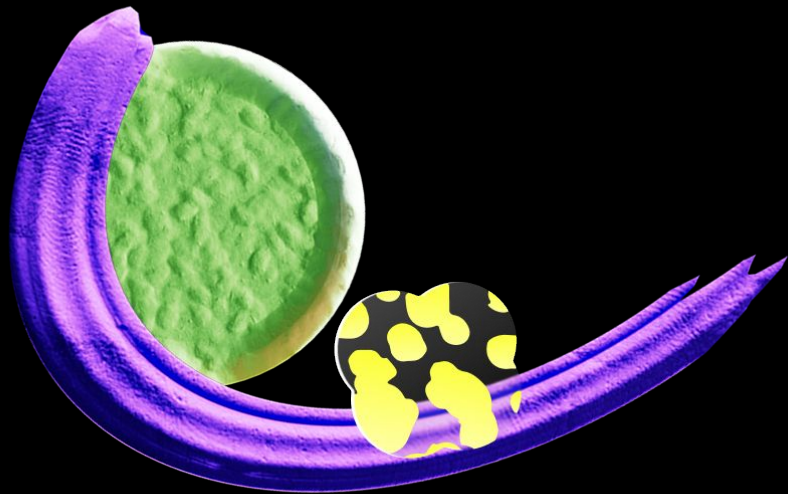
Отметьте студентов в журнале

Эволюционная архитектура и технический долг

Михаил Каморин
Tech Lead



Speaker



Михаил Каморин
Tech Lead

16 лет в IT, 6 лет вёл
разные курсы в OTUS

Сегодня вы узнаете, как:

- 1 **Использовать принципы эволюционной архитектуры**
- 2 **Оценивать технический долг и управлять им**
- 3 **Встраивать механизмы адаптации и гибкости в архитектуру**



План занятия

- 1 Принципы эволюционной архитектуры
- 2 Технический долг и архитектурные компромиссы
- 3 Стратегии адаптивной архитектуры
- 4 Архитектурная документация в условиях изменений

I. Принципы эволюционной архитектуры

01 Fitness functions

02 Принцип обратимости архитектурных решений



Эволюционная архитектура

- Ключевые идеи:
 - Адаптивность и гибкость компонентов
 - Контроль качества через измеримые метрики
 - Обратимость изменений архитектуры

Fitness functions

- Фитнес-функция – метрика, определяющая степень соответствия архитектуры выбранной цели
- Ключевые требования
 - возможность непрерывной измеримости
 - возможность блокировки выпуска релиза при несоблюдении фитнес-функций

Примеры фитнес-функций

- Время отклика API в 95% случаев составляет не более 200 мс
- Сервис обрабатывает 1000 запросов в секунду без деградации
- У каждого модуля не более 3 зависимостей от других модулей
- Модуль не содержит уязвимостей безопасности
- 80% кода покрыто тестами

Принцип обратимости

- Любое изменение может быть обратимо без разрушения системы
- Примеры реализации:
 - Версионирование API
 - Временные фасады или адаптеры для интеграции
 - Декомпозиция сервисов на микросервисы

Примеры вопросов

- Какие компоненты не могут быть заменены без влияния на всю систему?
- Могут ли изменения в одном модуле “протечь” в остальные части системы?
- Есть ли метрики отслеживания модульности?

Вопрос для общего обсуждения

Какая связь между BDD и фитнес-функциями?

2 минуты



Вопрос для общего обсуждения

Приведите пример обратимости изменений на уровне СУБД

3 минуты



II. Технический долг и архитектурные компромиссы

- 01 Архитектурный технический долг
- 02 Решения в условиях неопределённости
- 03 Техники борьбы с долгом



Технический долг – требования

- Видимость
- Измеримость
- Контролируемость

Архитектурный технический долг

- Сильное зацепление модулей
- Отсутствие версионирования API
- Недостаточная документация
- Сложные интеграции без тестов

Карта технического долга

- Классификация – архитектурный, инфраструктурный, организационный
- Причина возникновения долга
- Риск, который несёт неустранённый долг
- Предложения по снижению

Решения в условиях неопределённости

- “Временные опоры” – временное решение, которое можно **безопасно** убрать впоследствии
- Отложенное обязательство – откладываем критическое архитектурное решение до появления определённости

Техники борьбы с техническим долгом

- Архитектурный рефакторинг – не переписываем систему полностью, а постепенно изменяем, сохраняя функциональность
- Эволюционные улучшения – внедряем новые решения параллельно со старой архитектурой
- Feature slicing – разделение на модули по функциям (например, BDD-фичи)

Вопрос для общего обсуждения

Почему важно иметь карту технического долга?

2 минуты



Вопрос для общего обсуждения

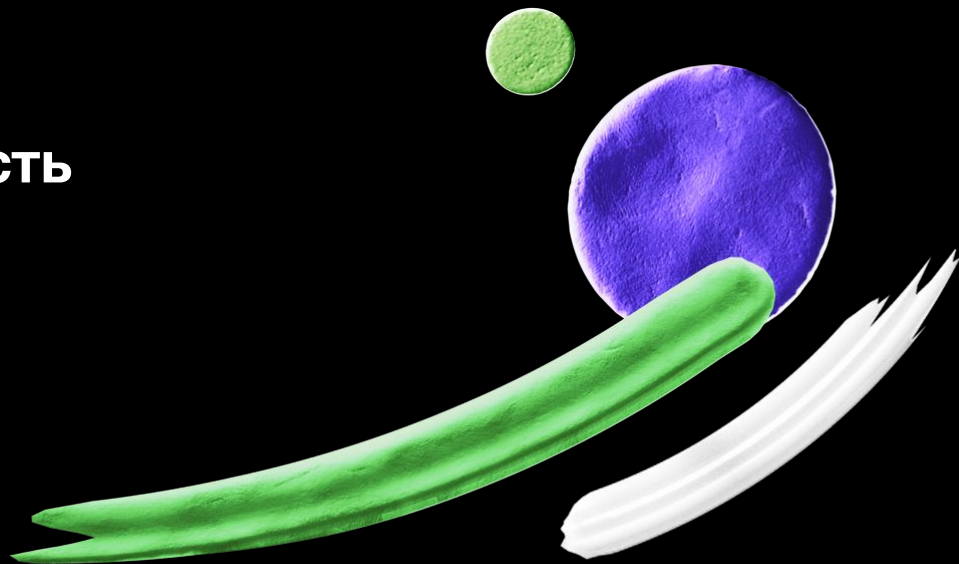
Можно ли полностью избежать наличия технического долга? Почему?

3 минуты



III. Стратегии адаптивной архитектуры

- 01 Гибкость через модульность
- 02 Architecture Control Layer
- 03 Стратегии эволюции API



Гибкость через модульность

- Ключевое свойство адаптивной архитектуры – **модульность**
 - Понятные границы ответственности компонентов
 - Замена или расширение отдельных компонентов не затрагивает систему в целом
 - Параллельная разработка нескольких функций и/или интеграций

Примеры модульности

- **Плагины** – модули, которые не несут самостоятельной бизнес-ценности, но являются изолированным функционалом
- **“Временные опоры”** – интерфейсы, фасады или адаптеры, которые нужны на время адаптации системы под новые требования

Architecture Control Layer

- Прослойка, которая управляет взаимодействием компонентов и ограничивает распространение изменений
 - Адаптеры
 - Anti-Corruption Layer
 - Фасады
 - API-шлюзы и Backend for Frontend

Стратегии эволюции API

- Версионирование
- Депрекация
- Обратная совместимость

Вопрос для общего обсуждения

Гарантирует ли версионирование API обратную совместимость?

2 минуты



IV. Архитектурная документация в условиях изменений

- 01 Документирование эволюции
- 02 ADR как основной инструмент
- 03 C4-модели как отражение текущего состояния



Документирование эволюции

- Архитектурный журнал:
 - Какие решения были приняты
 - Какие альтернативы рассматривались
 - Почему выбрали именно этот вариант
 - Какие последствия ожидалось

ADR как основной инструмент

- Полезные практики:
 - Версионирование решений
 - Кросс-ссылки между ADR
 - Явная фиксация устаревания
 - Минимальный размер

C4-модели как текущее состояние

- Документировать только значимое
- Model as Code (в идеале генерировать или проверять модели автоматически)
- Обновление модели как часть процесса изменений
- Разные уровни – разные задачи

ADR + C4: почему и что сейчас

- ADR объясняет причину появления
- C4 показывает местонахождение в системе и текущие взаимодействия
- Без ADR получаем “картинку без смысла”
- Без C4 ADR оторваны от реальности

Алгоритм обновления архитектуры

1. Анализируем влияние
2. Обновляем или создаём новый ADR
3. Обновляем C4
4. Проверяем фитнес-функции
5. Фиксируем последствия

Типичные ошибки

- Документация “ради галочки”
- Слишком подробные модели
- Отсутствие **связей** между артефактами
- Игнорирование устаревших решений

Вопрос для общего обсуждения

Какое самое важное требование прозвучало в последнем блоке?

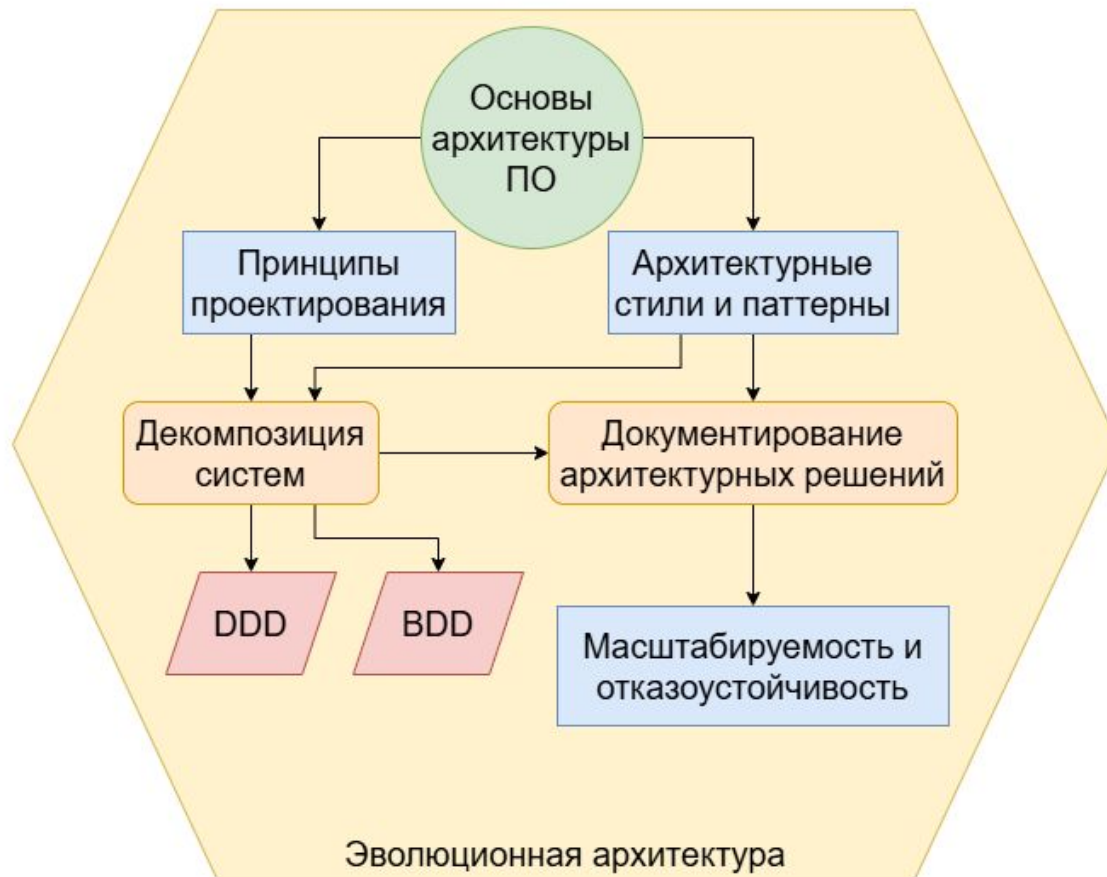
2 минуты



Сегодня мы узнали, как:

- 1 **Использовать принципы эволюционной архитектуры**
- 2 **Оценивать технический долг и управлять им**
- 3 **Встраивать механизмы адаптации и гибкости в архитектуру**





Вопросы?